

Django 靜態文件與 Nginx 容器掛載解析

Nginx 容器中的 volumes 配置作用

在您的 docker-compose.yml 中，Nginx 服務的 volumes 配置：

```
volumes:
  - static_volume_poa:/app/staticfiles/poa
  - static_volume_llm:/app/staticfiles/llm
```

這些配置在 Nginx 容器中實現以下功能：

1. 掛載 Docker 卷到指定路徑：

- 將 `static_volume_poa` 這個 Docker 卷掛載到 Nginx 容器內的 `/app/staticfiles/poa` 路徑
- 將 `static_volume_llm` 這個 Docker 卷掛載到 Nginx 容器內的 `/app/staticfiles/llm` 路徑

2. 實現靜態文件共享：

- 這些卷與 Django 應用容器共享，連接了兩個系統
- 當 Django 執行 `collectstatic` 命令時，靜態文件被收集到 `staticfiles` 目錄
- 通過 Docker 卷，這些文件同時對 Nginx 容器可見

3. 分離不同應用的靜態文件：

- 每個應用（`poa` 和 `llm`）的靜態文件放在不同目錄，防止命名衝突
- 配合 Django 中 `STATIC_URL = '/static/poa/'` 的設置使用

工作流程

1. Django 設置文件中指定了靜態文件收集位置：`STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')`
2. 當運行 `collectstatic` 命令時，Django 從 `STATICFILES_DIRS` 收集文件到 `staticfiles` 目錄
3. Docker 容器通過卷將這個目錄與 Nginx 容器共享
4. Nginx 配置文件中應該有對應的 `location` 指令來提供這些靜態文件服務

這樣設計的好處是實現了職責分離：Django 負責應用邏輯，Nginx 負責高效地提供靜態資源，提高了整體系統效能。

Nginx 靜態文件配置的工作原理

能讀取兩個靜態目錄的原因

在您的設置中，雖然 Nginx 設定只有一個 `/static/` 路徑的規則，但它確實可以讀取到兩個不同的靜態目錄，這是因為：

1. 路徑解析機制

```
location /static/ {  
    alias /app/staticfiles/;  
}
```

當 Nginx 收到一個 `/static/` 開頭的請求時，它會：

- 把 URL 中的 `/static/` 部分替換為 `/app/staticfiles/`
- 保留 URL 中的剩餘部分，這是關鍵

2. 目錄結構與 URL 對應

在您的 Docker Compose 設定中：

```
volumes:  
  - static_volume_poa:/app/staticfiles/poa  
  - static_volume_llm:/app/staticfiles/llm
```

這創建了以下目錄結構：

```
/app/staticfiles/  
├─ poa/ (來自 static_volume_poa)  
└─ llm/ (來自 static_volume_llm)
```

3. 請求流程示例

- 當瀏覽器請求 `/static/poa/css/style.css`：
 - Nginx 將其轉換為 `/app/staticfiles/poa/css/style.css`
 - 該文件存在於 `static_volume_poa` 卷中
- 當瀏覽器請求 `/static/llm/js/main.js`：
 - Nginx 將其轉換為 `/app/staticfiles/llm/js/main.js`
 - 該文件存在於 `static_volume_llm` 卷中

4. 搭配 Django 設定

Django 配置 `STATIC_URL = '/static/poa/'` 確保 POA 應用的所有靜態文件引用都帶有 `/static/poa/` 前綴，使它們能夠正確映射到 `/app/staticfiles/poa` 目錄。

LLM 應用也應該有類似的配置，使用 `/static/llm/` 作為 URL 前綴。

這種設計非常簡潔，一條 Nginx 規則就能處理來自兩個不同應用的靜態文件請求，無需為每個應用配置單獨的 location 區塊。

Django 應用的靜態文件 URL 前綴設置說明

STATIC_URL 設定的作用和意義

當您在兩個不同的 Django 應用中分別設定：

```
# POA 應用中的設定
STATIC_URL = '/static/poa/' # URL prefix for static files

# LLM 應用中的設定
STATIC_URL = '/static/llm/' # URL prefix for static files
```

這些設定具有以下作用：

1. 靜態文件 URL 的生成規則

- 當 POA 應用中使用 `{% static 'css/style.css' %}` 模板標籤時，Django 會生成 URL：`/static/poa/css/style.css`
- 當 LLM 應用中使用 `{% static 'js/main.js' %}` 模板標籤時，Django 會生成 URL：`/static/llm/js/main.js`

2. 與 Docker 卷映射的配合

這與您的 Docker 卷映射正好對應：

```
volumes:
  - static_volume_poa:/app/staticfiles/poa
  - static_volume_llm:/app/staticfiles/llm
```

當 Django 執行 `collectstatic` 命令時：

- POA 應用的靜態文件會收集到 `staticfiles` 目錄下
- LLM 應用的靜態文件會收集到 `staticfiles` 目錄下
- 這些文件會通過 Docker 卷映射到 Nginx 容器的相應路徑

3. 與 Nginx 配置的協同工作

Nginx 的配置：

```
location /static/ {
    alias /app/staticfiles/;
}
```

當瀏覽器請求 `/static/poa/xxx.css` 時：

- Nginx 將請求映射到 `/app/staticfiles/poa/xxx.css`
- 該文件存在於 `static_volume_poa` 卷中

當瀏覽器請求 `/static/llm/yyy.js` 時：

- Nginx 將請求映射到 `/app/staticfiles/llm/yyy.js`
- 該文件存在於 `static_volume_llm` 卷中

4. 分離不同應用的靜態文件

這種設計有效地將兩個應用的靜態文件隔離，即使它們有相同名稱的文件也不會衝突，因為它們位於不同的子目錄中。

這是一種優雅的解決方案，讓不同的應用可以獨立管理靜態文件，同時使用單一的 Nginx 配置來提供這些文件的訪問。